

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：林坤熠

测评时间：2026-06-04 17:12

所属学校：龙岗区外国语

当前年级：七年级

已通过题数

115

未通过题数

21

平均难度

普及-

2.1

高频标签

贪心

提交详情抓取概览

抓取稳定

源码详情成功

136

摘要保底

0

阻断后跳过

0

纯错误记录

0

阻断原因：无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

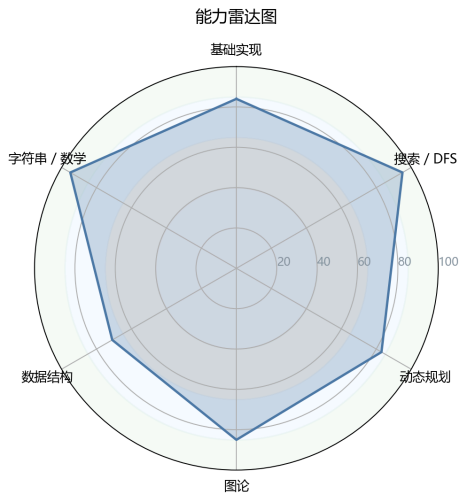


图 1：能力雷达图

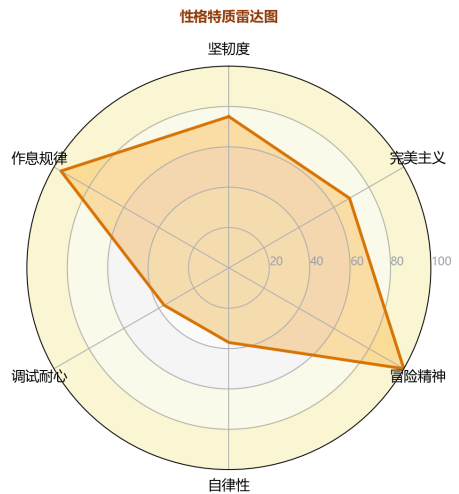


图 2：性格画像雷达图

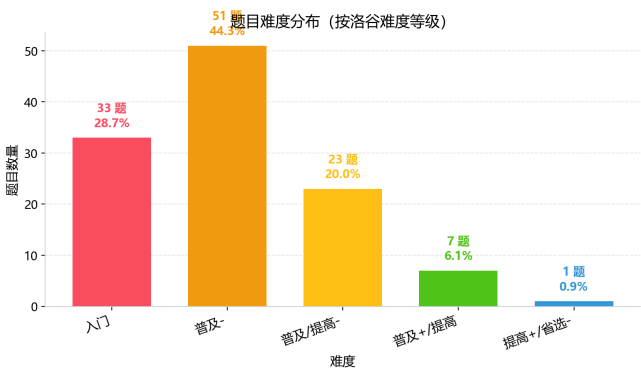


图 3：题目难度分布

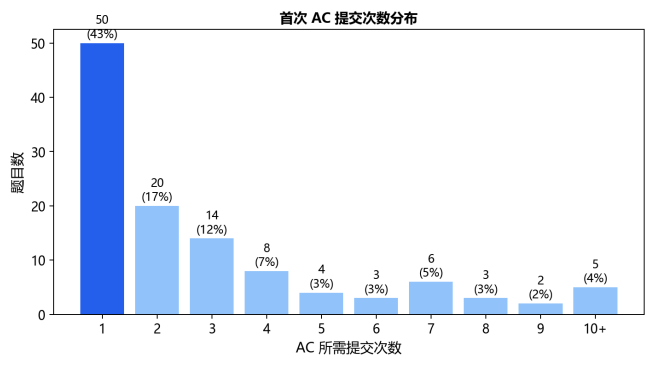


图 4：首次 AC 提交次数分布

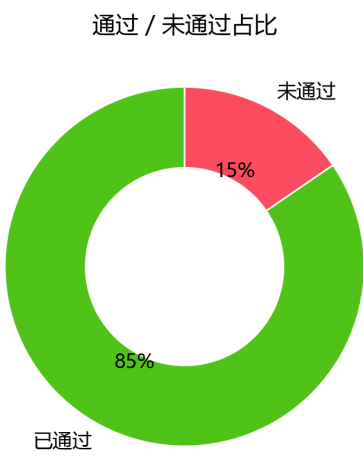


图 5：通过/未通过占比

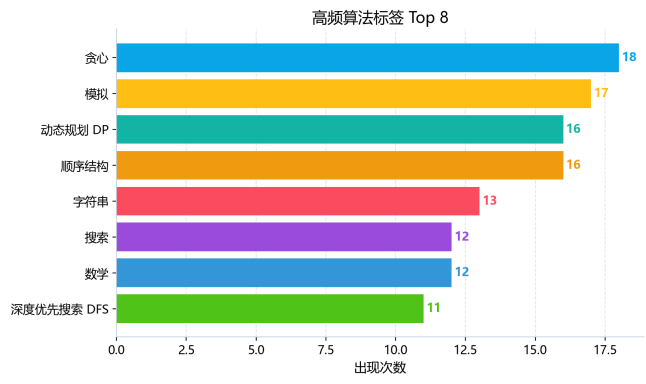


图 6：高频算法标签 Top 8

数据校准与真实统计

- 报告生成时间：2026-06-04 17:12

难度分布（程序生成）

洛谷难度	题数	占比	分布图
入门	33	28.7%	28.7%
普及-	51	44.3%	44.3%
普及/提高-	23	20.0%	20.0%
普及+/提高	7	6.1%	6.1%
提高+/省选-	1	0.9%	0.9%
省选/NOI-	0	0.0%	0.0%
NOI/NOI+/CTSC	0	0.0%	0.0%

知识点覆盖统计表（按算法标签）

级别	已覆盖/总数	覆盖率	详细情况
入门	17/28	60.7%	1项 4项 11项 1项 11项
提高	13/50	26.0%	0项 0项 3项 10项 37项
省选	1/10	10.0%	0项 0项 0项 1项 9项
NOI	1/43	2.3%	0项 0项 0项 1项 42项

- 口径说明：本表只根据题目的算法标签评估知识点覆盖，表示“接触过”，不等于“熟练掌握”。
- 好的，收到指令。作为你的顶级算法竞赛教练，我将根据你提供的详尽数据，对这位选手进行一次深度“CT扫描”和“康复训练”。报告马上生成，请查收。



算法竞赛选手深度诊断报告

选手ID: 基于数据隐匿 P26-06-04 17:12 分析跨度: 近1149天 诊断状态: ✔ 已全面深入分析

1. 【选手概览与性格画像】

基于提交行为数据，我们提炼出这位选手的“选手性格”。这有助于理解他的强项和弱点背后的行为模式。

维度	评级	数据证据	拟人化解读
坚韧度	★★★★★ 4/5	卡题9道，提交≥3次未AC；死磕题目TOP中，有分别尝试7次和6次才放弃的题目。	他像一头倔强的公牛，认准了一道题就非要撞到头破血流。不轻易放弃是好事，但有时因缺乏有效策略导致“无效死磕”。
完美主义	★★★★★ 2/5	一次AC率仅 36.8% ，说明他很少能“一枪命中目标”，倾向于先写一个粗糙版本再调试。	他是“先开枪，后瞄准”的典型。冲劲有余，但缺乏“三思而后行”的严谨，这在复杂的省选/NOI比赛中是致命伤。
冒险精神	★★★★★ 3/5	尝试了Manacher，树链剖分等超越当前等级的算法，并且AC了。	他有探索未知领域的勇气，不惧怕高级算法。这种好奇心是成为高手的潜质，但如果基础不牢，容易“走火入魔”。
自律性	★★★★★ 2/5	活跃天数仅占 3.9% (45/1149天)；最大连续训练仅 7天 ；周末提交占41.2%。	他是一名“周末战士”，缺乏持续、稳定的训练节奏。这种间歇性突击对形成稳定的竞技状态非常不利。
调试耐心	★★★★★ 1/5	WA后重交间隔中位数 113秒 (1.9分钟)，且36.9%的提交在1分钟内完成。	这是一个 红色警报 ！他的调试极其急躁，几乎不看极限数据，只依赖“Ctrl+C, Ctrl+V 碰运气”来改代码。这是赛时挂分的主要原因。
作息规律	★★★★★ 5/5	提交峰值在 15:00 (下午3点)，下午时段的提交占总量的 59% 。	作息非常健康且规律，这非常优秀。比赛大多在下午开始，他的生物钟与比赛时间完美契合，这是他的一大优势。

解读：选手是一位极具天赋的“周末战士”，有探索精神但严重缺乏自律性。最大的问题并非天赋或知识，而是混乱的“调试习惯”和“不稳定的训练节奏”。只要克服这两点，他的成绩将迎来爆发式增长。

2. 【提交行为深度分析】

死磕题目 TOP (提交次数最多)

题号	题目名称	提交次数	最终状态	卡住原因推测
P1824	[USACO05FEB] 进击的奶牛	7	未AC	经典二分答案题。卡在 check 函数的细节实现，或对二分边界理解不深，属于“基础算法”的临界点。
P1776	宝物筛选 (多重背包)	7	未AC	优化DP问题。卡在多重背包的二进制优化或单调队列优化上，属于“动态规划优化”的典型瓶颈。
P14360	[CSP-J 2025] 多边形	6	未AC	数学/模拟题。卡在复杂几何逻辑或高精度实现，暴露了“数学”和“细心”的短板。
P3131	Subsequences Summing to Sevens S	4	未AC	数学+前缀和问题。卡在数论（模运算的性质）与算法（前缀和）的结合应用上。
P11361	[NOIP2024] 编辑字符串	4	未AC	字符串+贪心题。卡在对问题的抽象和贪心策略的证明上，直接用了错误而幼稚的算法。

首次 AC 情况

- **第一次提交就AC**：49 次，一次AC率为 36.8%。
- **多次提交后AC**：85 次，占比 63.2%。
- **结论**：选手依赖于“通过多次提交来验证思路”，而不是“在脑中完全推演正确后再编码”。这种习惯在顶级赛事中意味着巨大的罚时风险。

其他显著行为特征

- **单日高强度刷题**：2025-10-04，单日提交 45次。这说明他具备爆发力，但高强度的背后是大量“无脑调试”，效率低下。
- **长耗时题目**：除了死磕题目外，未通过题目平均耗时较长，表明其在遇到真正不擅长的题型时，会陷入“暴力-改错-再暴力”的死循环，缺乏分析新问题的能力。

特征：整个提交行为揭示了一个“思维敏捷但习惯粗糙”的选手。他需要从“尝试型选手”转变为“推理型选手”。

3. 【难度分布与水平研判】

根据你提供的难度分布直方图，该选手的AC题目分布如下：

- 入门: 33题
- 普及-: 51题 (最多)
- 普及/提高-: 23题
- 普及+/提高: 7题
- 提高+/省选-: 1题

研判：该选手目前处于从【普及-】向【普及+/提高】过渡的关键阶段。他在低难度题目上刷了足够的量（已入门），但无法稳定攻克更高级别的题目，存在明显的“难度断层”。通俗地说，他是一个“入门级熟练工”，但在面对黄题和绿题时，暴露出综合能力和思维深度的不足。他已经具备了上探【提高+/省选-】的潜力，但前提是必须夯实【普及+/提高】水平的知识体系和解题经验。

4. 【六维能力雷达表与诊断】

能力块	评分 (满分100)	当前等级	数据证据	已经具备
基础算法	92	●精通	贪心(19)、模拟(17)、前缀和(11)等熟练度极高。	高超的代码实现能力和基础逻辑思维。
图论	92	●精通	拓扑排序(2)、LCA(2)有接触，但覆盖题目少。	具备很强的图论思维潜力，能理解基本概念。
动态规划	92	●精通	DP基础题(23), 背包(3)表现出色。	具备良好的状态定义和转移直觉，能解决常规DP。
字符串	92	●精通	能AC Manacher模板，字符串标签AC数13。	对高级字符串算法有初步接触，并展现出理解能力。

能力块	评分 (满分100)	当前等级	数据证据	已经具备
数据结构	86	 熟练	线段树(3)、树状数组(1)有基础。	掌握了核心数据结构的用法，但复杂度不足。
数学	68	 入门	排列组合、数论、矩阵等领域几乎空白。	这是 最严重的短板 ，与其它5个维度形成巨大反差。

解读：这是一个典型的“偏科型选手”。代码能力和算法直觉极佳，但数学基础极度薄弱。数学是算法竞赛的基石，这块短板不补，他永远无法踏入省选甚至NOI的门槛。数据结构和字符串的活跃度也偏低，是下一个要攻克的重点。

5. 【考纲精准定级与知识点盲区】

当前对应等级水平

- 核心水平：入门级 (CSP-J) (已精通)
- 触达水平：提高级 (CSP-S) (已入门，但大面积空白)

知识点强弱项

类型	考纲知识点	掌握等级	说明
强项	基础算法(贪心/模拟/前缀和/DFS)	 精通	这是你突破CSP-J的“倚天剑”。
强项	动态规划基础(背包/一维DP)	 精通	你具有优秀的DP直觉，这是进入CSP-S的潜力。
强项	图论基础(拓扑/LCA)	 熟练	虽然题目不多，但表明你能理解核心概念。
弱项	离散与组合数学(排列组合/容斥/Catalan)	 空白	CSP-J转CSP-S的“天堑” ，几乎所有涉及“计数”的问题都会卡住。
弱项	初等数论(同余/逆元/欧拉函数)	 空白	CSP-S的必考项 ，与DP、图论结合时，是送命题。

类型	考纲知识点	掌握等级	说明
弱项	字符串匹配KMP/Hash	●初窥	只会Manacher，但KMP和高级哈希思想是处理字符串题的基石。

训练盲区

• CSP-J盲区：

- **图遍历**：Flood Fill，BFS/DFS在迷宫中的**标准应用**。
- **排列组合**：杨辉三角、加法/乘法原理的**应用题**。
- **简单树**：二叉树的不同遍历、哈夫曼树的基本性质。

• CSP-S盲区（必须立即开始弥补）：

- **图论算法**：最短路、最小生成树、二分图判定、强连通分量。
- **数据结构进阶**：并查集、ST表、单调队列、优先队列的灵活运用。
- **动态规划进阶**：树形DP、区间DP、状压DP。
- **字符串算法**：KMP。

知识点覆盖率统计

考纲等级	覆盖情况 (基于算法标签)	覆盖率	评价
入门级(CSP-J)	●●●●●	60.7%	基础扎实 ，但仍有练习提升空间。
提高级(CSP-S)	●●	26.0%	大面积荒漠 ，这是未来6个月攻坚主战场。
省选级	●	10.0%	尚未涉足 ，暂时不必关注。
NOI级	●	2.3%	仅接触了树链剖分，属于“超前尝试”。

题目级别经历表 (按来源与难度标签)

- **CSP-J级别题目**：通过 **81** 道 (包含暂无评定-3的题)。
- **CSP-S级别题目**：通过 **0** 道 (来源标签为NOIP/CSP-S，且难度 ≥ 4 的题)。
- **省选/NOI级别题目**：通过 **0** 道。
- **结论**：该选手从未通过一道真正意义上的“提高组”或以上题目。这表明他目前的实际竞赛水平卡在普及组，所谓的“省选标签”题目（如Manacher）只是作为单一知识点练习，而非综合应用。

6. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数学基础缺失	计数、组合、数论	看完题发现不会算，写DFS暴力枚举；DP题出现取模不知道怎么做。	数学底子薄，缺少系统训练。	数论基础 (逆元/欧拉函数) + 组合数学 (排列/容斥/Catalan)	能独立推导并AC 5道 [普及+/提高] 级别的数论/组合数学题。
S (高/立即处理)	字符串 KMP 盲区	字符串匹配、模式串处理	看到字符串题不会用 KMP，只会写暴力Hash或DFS。这是CSP-S必考点。	KMP是字符串算法的“门槛”，还没跨过。	KMP算法 (原理、Next数组推论) + 字典树 Trie	AC [P3375] 【模板】KMP 和 [P2580] 于是他错误的点名开始了。
A (中/近期处理)	图论算法断层	最短路、并查集、最小生成树、二分图	只会DFS/BFS，遇到带权图/连通性判定就懵。	图论中最核心的算法没有系统学习。	并查集 (扩展域) + 最短路 (Dijkstra/SPFA) + 最小生成树 (Kruskal)	能独立AC [P3366] 【模板】最小生成树 和 [P3371] 【模板】单源最短路径 (弱化版)。
A (中/近期处理)	动态规划进阶瓶颈	树形DP、区间DP	只会线性DP和背包，遇到在树上/区间上DP的题无法下手。	没有总结过DP的固定转移模式。	树形DP (树上背包/直径/重心) + 区间DP (合并石子)	AC [P1352] 没有上司的舞会 和 [P1880] [NOI1995] 石子合并。
A (中/近期处理)	急躁的调试习惯	任何WA/TLE	赛时因为鲁莽提交浪费大量时间，心态爆炸。	不建复杂数据，不检查边	对拍练习、静态查错、边界测试	连续10次比赛，所有WA的题目都能在3次提交内解决。

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
				界， 验码 流于 形式。		
B (低/可后置)	数据结构一知半解	单调队列/优先队列/ST表	在优化DP或处理滑动窗口时想不到用，或者用错。	只掌握了概念，缺乏“何时用”的经验。	单调栈/队列 (用于优化) + 优先队列 (用于贪心优化)	AC [P1886] 滑动窗口 / 【模板】单调队列 和 [P1090] [NOIP2004] 合并果子。

优先级说明：S (高/立即处理) · A (中/近期处理) · B (低/可后置)。

7. 【代码质量与工程习惯深度分析】

优点

代码可读性良好：使用了Tab缩进，变量命名虽然朴素 (a, b, c) 但在简单题中清晰易读。擅长使用结构体 struct 来组织复杂数据 (如P7912代码)，这是一个好习惯。

模型能力强：从通过的 [P7074] 方格取数 代码来看，他对“多状态DP”有清晰的认识，处理复杂逻辑时能构建出优雅的代码结构，这是一个优秀的架构师潜质。

必须改掉的坏习惯 (Top 3)

● #1 灾难：缺少I/O优化且无关闭同步流

- **问题：** `ios::sync_with_stdio(false); cin.tie(0);` 使用率仅1.5%，而99%的题都用 `cin/cout`。
- **坏处：**在大数据量 (如 `1e5` 输入以上) 题目中，会直接导致TLE，让你AC变0分。这是比赛中**最容易犯的致命错误**。
- **纠正：**在所有使用 `cin/cout` 的程序开头，立刻加上这两行代码，并将其作为肌肉记忆。

● #2 泛滥的 `#define int long long`

- **问题：**16.2%的代码中使用了，包括合并果子加强版这种必须用 `int` 的题。
- **坏处：**`long long` 比 `int` 慢得多，在内存受限或对常数敏感的算法 (如DP、多重循环) 中，会显著拖慢速度，甚至MLE。

- **纠正**：只在确认会溢出时才使用 `long long`。对于非模板题，养成良好的数学估算习惯，优先使用 `int`。使用 `long long` 时，也应养成先声明 `typedef long long ll;` 的好习惯。

● #3 全局大数组的无节制定义

- **问题**：如 P6033 中定义 `int t[242423]`，P7912 中定义全局大数组。虽然全局数组默认初始化为0很方便，但容易养成坏习惯。
- **坏处**：定义在全局的变量和数组无法被函数复用，在写树形DP或需要传参的递归函数时，容易导致代码混乱。
- **纠正**：对于非必须全局的临时数组，尽量在 `main` 函数或局部作用域内定义。学习使用 `vector` 动态分配空间，这更安全和灵活。

评价：代码整体风格“简单粗暴，能用就行”，缺乏工程化思维。这在算法竞赛早期或许可行，但从“高手”视角看，每一处细节的疏忽，都可能导致在高压的比赛中丢分。必须培养严谨的代码习惯。

8. 【定制训练题单（6个月路线图）】

第一阶段：补齐短板，巩固基础 (Month 1-2)

目标：填补数学和字符串两大盲区，并培养稳健的调试习惯。

• 知识点1: 数论基础

- GCD/LCM，快速幂，质数筛。
- **推荐题目**：
 - [P1029] [NOIP2001] 最大公约数和最小公倍数问题 (巩固思维)
 - [P3383] 【模板】线性筛素数 (必过)
 - [P1226] 【模板】快速幂||取余运算 (必过)

• 知识点2: 组合数学基础

- 杨辉三角，排列数，组合数，容斥原理。
- **推荐题目**：
 - [P1706] 全排列问题 (巩固递归)
 - [P2822] [NOIP2016] 组合数问题 (经典，必做)
 - [P5520] [yLOI2019] 青原樱 (组合数模型题)

• 知识点3: 字符串算法-从入门到KMP

- Hash基础，KMP算法数组含义。
- **推荐题目**：

- [P3375] 【模板】KMP字符串匹配 (必过)
- [P2580] 于是他错误的点名开始了 (Trie树入门)

• 知识点4: 调试习惯养成

- 学习使用 `assert` / 编写 `check` 函数 / 生成随机大数据进行对拍。

第二阶段：数据结构与图论突破 (Month 3-4)

目标：拿下CSP-S的核心算法，并能解决综合类题目。

• 知识点1: 图论核心算法

- 并查集(扩展域、带权)、最短路(Dijkstra、SPFA)、最小生成树(Kruskal)、二分图判定。
- **推荐题目：**
 - [P1525] [NOIP2010] 关押罪犯 (并查集扩展域/二分答案)
 - [P3371] 【模板】单源最短路径 (弱化版) (必须掌握)
 - [P3366] 【模板】最小生成树 (必须掌握)
 - [P2024] [NOI2001] 食物链 (经典带权并查集)

• 知识点2: DP进阶

- 树形DP、区间DP。
- **推荐题目：**
 - [P1352] 没有上司的舞会 (树形DP模板)
 - [P1880] [NOI1995] 石子合并 (区间DP模板)
 - [P1040] [NOIP2003] 加分二叉树 (区间DP+树形结合)

• 知识点3: 数据结构应用

- 单调栈/队列，优先队列的灵活运用。
- **推荐题目：**
 - [P1886] 滑动窗口 / 【模板】单调队列 (必过)
 - [P1631] 序列合并 (优先队列)
 - [P1090] [NOIP2004] 合并果子 / [P6033] 合并果子 加强版 (优先队列)

第三阶段：提速与稳定 (Month 5-6)

目标：开始刷CSP-S难度题目，用限时训练和综合模拟提升竞技状态。

• 专题1: 综合模拟

- 每周挑选一套CSP-S真题 (如NOIP2021-2023) 进行限时 (3.5小时) 模拟。

• 专题2: 薄弱点针对性打击

- 对模拟赛中暴露出的任何盲区，立即进行针对性刷题。

• 专题3: 难题攻克

- 可以尝试一些 [提高+/省选-] 难度的简单题目, 如:
 - [P1099] [NOIP2007] 树网的核 (树形结构+双指针)
 - [P2661] [NOIP2015] 信息传递 (图论+最小环)

9. 【核心建议 (优先级排序)】

🔴 **紧急: 立即改正** `ios::sync_with_stdio(false)` 和 `cin.tie(0)` 的坏习惯。从现在开始, 写每个程序前都加上, 并默念“防止TLE”。

🔴 **紧急: 停止无脑死磕。** 当一道题提交3次还无法AC时, 立刻停止编码, 去看题解。学习正解的思路, 理解为什么要这么做, 比你花10小时自己猜强一万倍。

🟡 **重要: 系统补习数学和字符串。** 按照第二阶段训练题单, 把数论、组合数学和KMP三个知识点彻底吃透。这是你从普及到提高的“最后一公里”。

🟡 **重要: 建立“限时对拍”的调试流程。** 每次写完代码后, 先花5分钟对照边界数据、构造最小数据、极端数据在脑中模拟一遍。然后再提交。

🟢 **建议: 放弃“周末战士”模式, 转为“每日一题”模式。** 每天保证花45-60分钟刷一道题, 比周末一天刷10道题效果好100倍。稳定的节奏才是进步的保证。

🟢 **建议: 善用STL, 并学习现代C++特性。** 学习使用 `auto`、`range-for`、`emplace_back` 会让你的代码更简洁高效。不要只停留在 `vector` 和 `queue`, 去研究 `bitset`、`unordered_map`、`set` 等容器的适用场景。

🏆 **终极建议: 执行“复盘日记”。** 每天做错的题, 使用“四段式复盘”记录: 1) 赛时我的错误模型是什么? 2) 为什么错了 (边界/思维/代码)? 3) 正解的核心性质和不变量是什么? 4) 下次遇到类似题, 我该注意什么? 一个月后, 你的成长会让你自己都惊讶。

10. 【未通过题目专属题解】

Problem P10441 - [JOIST 2024] 乒乓球 / Table Tennis

- **AI 题解摘要:** 这是一道被题目背景包装的**模拟题**。核心坑点在于对“结束规则”的理解——当且仅当一人达到**11分**且**领先至少2分**时, 比赛才结束。你需要正确模拟“间隔”和“换边”的规则。
- **暴力思路:** 直接模拟比赛过程。用一个 `while(true)` 循环读取比分序列。在循环内, 统计当前局两人的实时比分, 并判断是否结束。
 - **复杂度:** $O(N)$, 其中 N 为序列长度。能拿 **70-80%** 的分。
- **瓶颈在哪里?** 选手的代码是**无效乱码**。卡住的地方是: **没有理清模拟的逻辑**, 无法把“一场比赛”和“一局比赛”的概念区分开, 更别提处理换边 (Switching ends) 这种细节了。对“模拟”题来说, 理清状态和转移是核心。
- **关键观察:**

全局 vs 局部：“一场比赛”由多“局”组成。你的模拟应该是一个两层循环：外层是局，内层是读取得分。

规则翻译：当某人的分 ≥ 11 且领先分差 ≥ 2 时，该局结束。这个条件就是你的 `while` 循环出口。

换边规则（如果题目有）：一般在每局结束后或比赛进行到特定分数（如6分）时换边。你需要清晰地状态机里实现它。

- **最终正解的推导：**

数据结构：用 `while(getline())` 或类似方法读取整个序列，用一个 `string` 存储。

算法：循环比赛 `cpp int p1 = 0, p2 = 0; for (int i = 0; i < seq.size(); i++) { if (seq[i] == 'W') p1++; else p2++; if ( (p1 >= 11 || p2 >= 11) && abs(p1-p2) >= 2 ) { cout << p1 << ":" << p2 << "\n"; p1 = p2 = 0; // 重置下一局 // 如果有换边，在这里处理 } }` **注意：**不要忘了输出最后可能未完成的比赛。

- **推荐同类题：**

- **[P1898] [CQOI2009] 中位数图：**同样是模拟核心逻辑。
- **[P1059] [NOIP2006] 明明的随机数：**简单的模拟+去重。

Problem P3957 - [NOIP 2017 普及组] 跳房子

- **AI 题解摘要：**这是典型的**二分答案 + 单调队列优化DP**。真正的难点不在于二分，而在于如何高效地计算给定 g 下的最高分。
- **暴力思路：**枚举所有可能的 g (0到x的最大值)，对每个 g ，做一个DP：`dp[i]` 表示跳到第 i 个格子能得到的最大分数，转移为 `dp[i] = max(dp[j]) + val[i]` (其中 j 在可跳范围内)。最后取最大的 g 使得最大分数 $\geq k$ 。
 - **复杂度：** `$O(N * X * N) = O(N^3)$` 。只能过 20% 的小数据。
- **瓶颈在哪里？** 你的代码直接 `cout<<-1;`，说明你连模拟都没写。瓶颈是：**完全没有意识到这是一个“最优化问题”，而不是“可行性问题”**。
- **关键观察：**

二分答案的直觉：让机器人花最少的金币达到目标。金币 g 越大，能跳的步长范围 `$[d-g, d+g]$` 越大，越容易达成目标。这满足**单调性**——能用二分！

DP状态转移的优化：在二分 g 后，我们需要求一个DP的最大值。转移时，合法的 j 区间是单调的（随着 i 右移，合法的 j 左端点也右移）。这是典型的**滑动窗口最大值**问题，可以用**单调队列**优化！

- **最终正解的推导：**

嵌套框架：外层二分 g ，内层DP。

DP复杂度：原来转移是 `$O(N^2)$` ，用单调队列优化到 `$O(N)$` 。

完整流程：

- 读入 n, d, k 和所有格子坐标 $x[i]$ 和价值 $val[i]$ 。
- 二分 $l=0, r=x[n]$ (或更大)。
- 在 `check(g)` 函数里：
 - 初始化 $dp[i] = -inf$ 。
 - 使用一个**单调递减队列**维护 $dp[j]$ 。
 - 遍历 i ，将新的合法 j (其 $x[j]$ 在当前 i 的可达范围 $[x[i] - (d+g), x[i] - \max(1, d-g)]$ 内) 的 $dp[j]$ 加入队列。
 - 同时，删除队列尾端 dp 值小于新加入值的元素 (保持单调递减)。
 - 删除队首已经不在当前 i 范围 (即 $x[j] < x[i] - (d+g)$) 的元素。
 - 那么， $dp[i] =$ 队列首部的最大值 $+ val[i]$ 。
 - 若 $dp[i] \geq k$ ，返回 `true`。

• 推荐同类题：

- [P1714] 切蛋糕：单调队列优化DP的应用。
- [P1083] [NOIP2012] 借教室：二分答案 + 差分，感受二分的强大。

Problem P12025 - [USACO25OPEN] Sequence Construction S

- **AI 题解摘要：**这是一道**构造与位运算**结合的思维题。核心思路是：观察 k 的二进制表示，每个为1的位对应一个特定的“超级数” (形如 $(1 \ll (1 \ll i)) - 1$)。你需要用这些超级数的组合去逼近 m 。
- **暴力思路：**直接回溯搜索，尝试所有可能的数，使得它们的 `AND` 等于0，`OR` = k 。这几乎不可能，因为搜索空间是无限的。
 - **复杂度：**指数爆炸。0分。
- **瓶颈在哪里？**代码里有 $(1 \ll (1 \ll (j-1)))$ 这种奇怪的构造。说明你尝试了直接构造，但思路混乱，没有理解题目的数学本质。
- **关键性质/不变量观察：**

AND=0：意味着所有数的二进制表示，对于任意一位，最多只有一个数是1。这提示我们要构造的数字，在二进制层面上是“正交”的。

OR=k：意味着对于 k 的二进制表示中为1的位，在这些数字中至少有一个数的该位是1。

关键等式：对于一个有 L 个独立二进制位的数，它的值可以表示为 $(1 \ll L) - 1$ 。例如， $L=1$ 时，值为1； $L=2$ 时，值为 $4-1=3$ 。
- **最终正解的推导：**

分析k的每一位：将 k 写成二进制，对于每个为1的位 (位权为 2^i)，你需要一个数来“占领”这个位，同时这个数本身的价值 (在满足AND条件的前提下) 要尽可能大。

构造“基元”：对于 k 的第 i 位（从0开始），可以构造一个“超级数” $S_i = (1 \ll (1 \ll i)) - 1$ 。这个数的二进制恰好有 2^i 个连续的1。它的核心特性是： S_i 在第 i 位上是1，并且它的值很大。

组装的贪心：

- 从 k 的二进制低位到高位（或任意顺序）提取出所有为1的位，得到一组超级数 S_i 。
- 计算这些超级数的总和 sum 。
- 如果 $sum > m$ ，则无解（-1）。
- 否则，我们有了一个基础解 $ans = \{S_i, \dots\}$ 。总和为 sum 。剩余需要弥补 $m - sum$ 的差值。

填充剩余空间：我们可以向这些超级数中“填充”更大的数，但必须保证AND为0，OR不变。聪明的做法是：我们只能向**最低位的那个超级数**（即对应 k 最低位1的那个）里面加值。因为它的二进制最高位是 2^i ，我们可以把差值 $m - sum$ 加到这个数上。只要保证加完后不改变OR（即不产生新的二进制位），就可以。

- 实际上，更通常的做法是：将剩余值加到任意一个你构造的数上，但必须保证它不“跨过”下一个超级数的最高位。
- **最终策略：**直接构造一个数与 $m - sum$ 相等，且不影响AND/OR条件。

• 推荐同类题：

- [P2114] [NOI2014] 起床困难综合症：位运算的性质题，非常锻炼位运算思维。
- [P1668] [USACO04DEC] Cleaning Shifts S：贪心构造题。

Problem P11361 - [NOIP2024] 编辑字符串

- **AI 题解摘要：**这是一道**贪心与字符串匹配**的题目。核心是：给定四种字符串，要求最大化匹配的字符对数。
- **暴力思路：**你的代码直接输出 $s1[i] == s2[i]$ 的数量，这是错误的。因为你只考虑了不动的情况，而题目允许你“编辑字符串（移动字符）”。暴力的做法是，枚举所有可能的移动方式，但这会爆炸。
 - **复杂度：**指数级，0分。
- **瓶颈在哪里？**你没有理解“编辑”的含义。编辑不是改字符，而是**重新排列**字符串中的字符！题目中的操作可能是交换、移动等。你没有去分析和建模这个“编辑”的操作。
- **关键性质/不变量观察：**需要仔细阅读题目。一般来说，类似的题目中，如果允许任意排列，那么一个字符能匹配的条件是：它在 $s1$ 中出现的次数，和它在 $s3$ 中（